

Algoritmi e Strutture Dati–parte I

lez. 8

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 8.30-10.30
(aula 18, Terzo Piano)
- Venerdì 8.30-10.30 (aula 2, Primo Piano)

r

Ricevimento

- Orario: Mercoledì 14.00-18.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

- La Frequenza del corso non è obbligatoria

Modalità dell' esame

- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

Testi Consigliati

- ALGORITMI:
- Cormen, Leiserson, Rivest, Stein
*Introduzione agli Algoritmi e
alle strutture Dati*
(seconda edizione o terza edizione)
Ed. Mc Graw-Hill.

Random Quicksort: Caso Peggior

- Dimostriamo che il quicksort nel caso peggiore ha un tempo di esecuzione $\Theta(n^2)$. Utilizziamo il metodo di sostituzione per dimostrare che il tempo di esecuzione è $O(n^2)$.
- Sia $T(n)$ il tempo nel caso peggiore per la procedura QUICKSORT con un input di dimensione n . Allora la ricorrenza è:

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n - q - 1)) + \Theta(n)$$

dove il parametro q varia tra 0 e $n - 1$.

(cont.)

- Supponiamo che $T(n) \leq cn^2$ per qualche costante c . Sostituendo: $T(n) \leq \max_{0 \leq q \leq n-1} (cq^2 + c(n - q - 1)^2) + \Theta(n)$
$$\leq c \max_{0 \leq q \leq n-1} (q^2 + (n - q - 1)^2) + \Theta(n)$$
- Ora l' espressione $q^2 + (n - q - 1)^2$ è **massima nei due estremi dell' intervallo $0 \leq q \leq n - 1$**
- Perché ? (la derivata rispetto a q si annulla in un solo punto che è un minimo).

(cont.)

- Sostituendo al posto di q $(n - 1)$ abbiamo:

$$T(n) \leq c (n - 1)^2 + c(n - (n - 1) - 1)^2 + \Theta(n)$$

$$T(n) \leq c (n - 1)^2 + \Theta(n)$$

$$T(n) \leq cn^2 - c(2n - 1) + \Theta(n)$$

$$T(n) \leq cn^2$$

basta scegliere un c tale che $c(2n - 1) - \Theta(n)$ sia positivo.

(cont.)

- Utilizziamo il metodo di sostituzione per dimostrare che il tempo di esecuzione è $\Omega(n^2)$.
- I calcoli sono analoghi ottenendo:

$$T(n) \geq c(n-1)^2 + \Theta(n)$$

$$T(n) \geq cn^2 - c(2n-1) + \Theta(n)$$

$$T(n) \geq cn^2$$

basta scegliere un c tale che $\Theta(n) - c(2n-1)$ sia positivo.

- Quindi abbiamo dimostrato che **$T(n) = \Theta(n^2)$**

Valore atteso

- In teoria della probabilità il valore atteso (chiamato anche media o speranza matematica) di una variabile aleatoria (casuale), è un numero indicato con $E[x]$.
- Il valore atteso formalizza il concetto di valore medio di un fenomeno aleatorio.

Valore atteso (cont.)

- Il valore atteso di una variabile aleatoria è dato dalla somma dei possibili valori di tale variabile, ciascuno moltiplicato per la probabilità di essere assunto, cioè è la **media ponderata** dei possibili risultati.

Valore Atteso di variabili aleatorie continue

- Nel caso di una variabile aleatoria continua che ammette come funzione di probabilità $f(x)$, il valore atteso è:

$$E[x] = \int_{-\infty}^{+\infty} x f(x) dx$$

Valore Atteso di variabili aleatorie discrete

- Nel caso di una variabile aleatoria discreta, che ha funzione di probabilità p_i allora:

$$E[x] = \sum_{i=1}^{+\infty} x_i p_i$$

- La media è un caso particolare del valore atteso che si ottiene quando il numero dei valori è N e la probabilità per ogni valore è $p_i = \frac{1}{N}$.

Proprietà del valore atteso

- **Valore atteso di una costante**

- Il valore atteso di una variabile aleatoria costante è ancora la costante stessa: $E[c] = c$

- **Linearità**

- $E[aX + bY] = aE[X] + bE[Y]$ dove a, b sono costanti ed X, Y sono variabili aleatorie

Tempo di esecuzione atteso

- Quicksort e Randomized-Quicksort differiscono solo nel modo in cui viene scelto il pivot, per il resto sono identiche.
- Pertanto analizzeremo le procedure PARTITION e QUICKSORT assumendo che ogni volta il perno sia scelto in modo casuale.
- Il tempo di esecuzione di QUICKSORT è dominato dal tempo che impiega la funzione PARTITION.

(cont.)

- Ogni volta che viene selezionato un elemento pivot, questo elemento non verrà mai incluso nelle successive chiamate ricorsive di QUICKSORT e PARTITION.
- Quindi ci sono al più n chiamate ricorsive di PARTITION.
- Una chiamata di PARTITION impiega il tempo $O(1)$ più una quantità di tempo che è proporzionale al numero di iterazioni del ciclo for delle righe 3-6.

Partition(A, p, r)

1. $x \leftarrow A[r]$
2. $i \leftarrow p - 1$
3. **for** $j \leftarrow p$ to $r - 1$
4. **do if** $A[j] \leq x$
5. **then** $i \leftarrow i + 1$
6. scambia $A[i] \Leftrightarrow A[j]$
7. scambia $A[i + 1] \Leftrightarrow A[r]$
8. **return** $i + 1$

(cont.)

- Ogni iterazione di questo ciclo for effettua un confronto tra il pivot ed un elemento dell' array.
- Pertanto vale il seguente lemma.
- Lemma: Se X è il numero dei confronti svolti nella riga 4 di PARTITION nell' intera esecuzione di QUICKSORT su un array di n elementi, allora il tempo di esecuzione di QUICKSORT è $O(n + X)$.

Obiettivo: Calcolare X

- Non calcoleremo esattamente X , ma deriveremo un limite superiore sul numero globale dei confronti. Abbiamo pertanto necessità di capire quando viene effettuato un confronto tra due elementi dell' array.
- Rinominiamo $z_1, z_2, \dots, z_n$ gli elementi dell' array, dove z_i è l' i -esimo elemento più piccolo.
- Definiamo $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$, l' insieme degli elementi compresi tra z_i e z_j .

(cont.)

- Utilizzando una variabile indicatrice, definiamo $X_{ij} = I\{z_i \text{ è confrontato con } z_j\}$. Stiamo considerando tutti i confronti fatti dall' algoritmo, non solo durante un' iterazione o chiamata di PARTITION.
- Poichè una coppia viene confrontata al più una sola volta, il numero totale dei confronti, è dato da: $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$.

(cont.)

- Prendendo i valori attesi da entrambi i lati, otteniamo:

$$E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$$

- $E[X_{ij}]$ è dato dalla probabilità con cui z_i è confrontato con z_j .

(cont.)

- La nostra analisi suppone che ogni pivot sia scelto in modo casuale ed indipendente. È utile riflettere su quando due elementi non sono confrontati.
- In generale, una volta che viene scelto un pivot x con $z_i < x < z_j$, sappiamo che z_i e z_j non potranno essere confrontati in un istante successivo. Se viene scelto z_i come pivot prima di qualsiasi elemento di Z_{ij} sarà confrontato con tutti gli elementi tranne che con sè stesso.

(cont.)

- Analogamente, se viene scelto come perno z_j . Quindi z_i e z_j sono confrontati se e soltanto se il primo termine ad essere scelto come pivot sarà z_i o z_j .
- Poichè l'insieme ha $j - i + 1$ elementi, la probabilità che qualsiasi elemento sia scelto per primo come pivot è: $\frac{1}{j-i+1}$. Pertanto la probabilità di confronto tra z_i e z_j è $E[X_{ij}] = \frac{2}{j-i+1}$.

(cont.)

- Sostituendo otteniamo:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}$$

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$$

$$E[X] < \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} < \sum_{i=1}^{n-1} \log n$$

$$E[X] = \sum_{i=1}^{n-1} O(\log n) = O(n \log n)$$

(cont.)

- Pertanto il RANDOMIZED-QUICKSORT ha un tempo di esecuzione atteso è $O(n \log n)$, quando i valori degli elementi sono distinti.

Heapsort

Heapsort

- L' idea : Utilizzare una nuova struttura dati (**heap**) per ordinare un array
- Costo Computazionale: **$O(n \log n)$**
- Il concetto di Heap puo' essere utilizzato per implementare le **CODE CON PRIORITA'**

Albero Binario

- L' albero binario è definito in modo ricorsivo
- Una albero binario T è definito come un insieme finito di nodi che è: **vuoto o è composto da tre insiemi disgiunti di nodi:**
 - un **nodo radice**;
 - un albero binario (**sottoalbero binario destro**);
 - un albero binario (**sottoalbero binario sinistro**)

(cont.)

- Terminologia:
 - Radice sottoalbero S D--> figlio S e D
 - nodo radice ---> padre dei figli S e D;
 - nodi senza figli: **foglie**

Alberi Binari (cont.)

- La **profondità** di un nodo è la lunghezza del percorso dalla radice al nodo (i.e. **numero archi attraversati**)
- L' **altezza** di un nodo è l' inverso della profondità
- **Livello**: l' insieme dei nodi alla stessa altezza
- **Altezza dell' albero** = **altezza della radice**
- Il numero dei figli è detto **grado** del nodo.

Albero Binario Perfetto

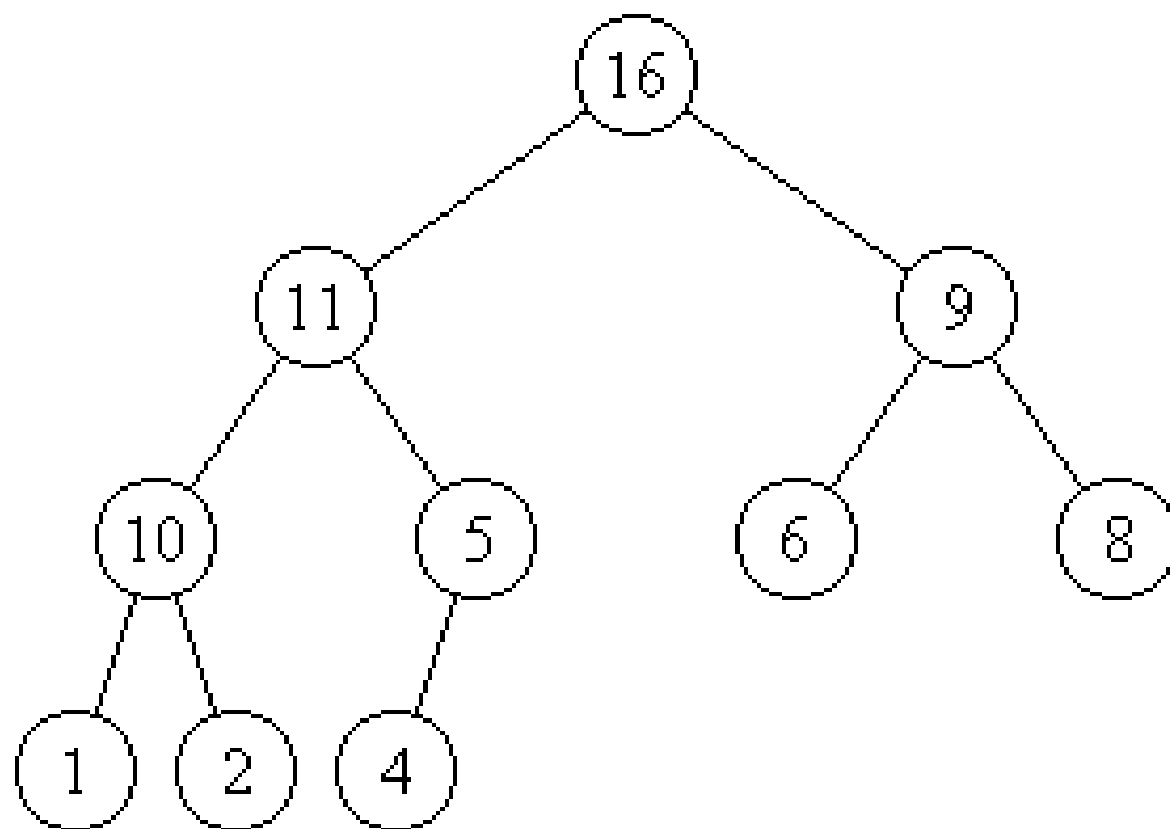
- Albero Binario Perfetto (**ABP**):
tutte le foglie hanno la stessa altezza h ;
i nodi interni hanno **grado 2**
- Altezza ABP: $\lfloor \log_2 n \rfloor$ (n numero nodi)
- Nodi ABP: Se ha altezza h il numero dei nodi è $2^{h+1} - 1$

Albero Binario Completo

- Tutte le foglie hanno profondità h o $(h - 1)$.
- Il numero dei nodi a livello $(h - 1)$ deve essere 2^{h-1} .
- Tutti i nodi a livello h sono accatastati a sinistra.
- Tutti i nodi interni hanno grado 2.

Albero Binari Heap

- Albero **Max-Heap**: se e solo se $A[\text{Parent}(i)] \geq A[i]$ ed è un albero binario completo
- Albero **Min-Heap** viceversa
- Un albero Heap implementa un ordinamento parziale: **riflessivo**, **antisimmetrico** (se $n \geq m$ e $m \geq n$ allora $n = m$) e **transitivo**.



Ordinamento Parziale

- Utili per modellare gerarchie complesse o mantenere informazioni parziali
- Nozione più debole dell' ordinamento totale ma più facile da costruire.

Array Heap

- Array A di lunghezza length ; Dimensione $\text{heapsize} \leq \text{length}$
- Come è organizzato ?
- $A[1]$ radice; $\text{Parent}(i) \leftarrow \lfloor i/2 \rfloor$
- $\text{Left}(i) \leftarrow 2i$; $\text{Right}(i) \leftarrow 2i + 1$
- Ricapitolando: $A[i] \geq A[2i]$ e $A[i] \geq A[2i + 1]$

Procedure per gestire un Heap

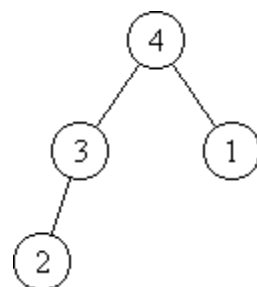
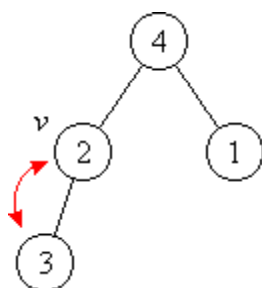
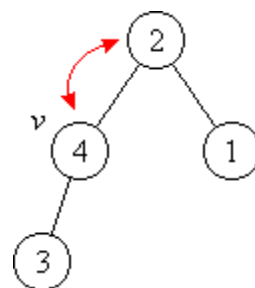
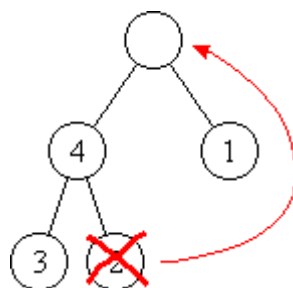
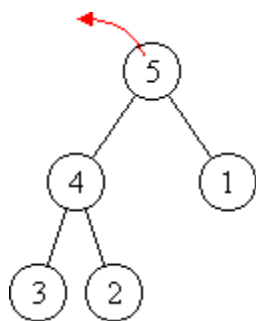
- La procedura **Max-Heapify**, di complessità $O(\log n)$, è un algoritmo che serve per mantenere la proprietà del Heap.
- La procedura, **Build-Max-Heap**, di complessità $O(n)$, è un algoritmo che costruisce un Heap da zero.
- La procedura **Heapsort**, di complessità $O(n \log n)$ è un algoritmo che ordina sul posto un array.

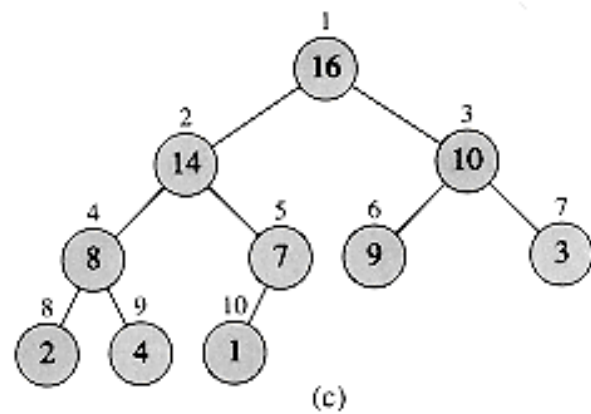
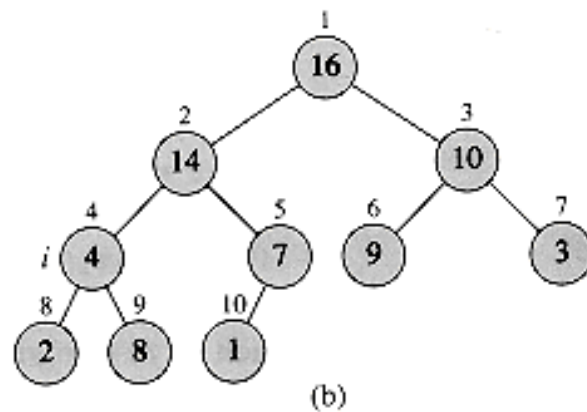
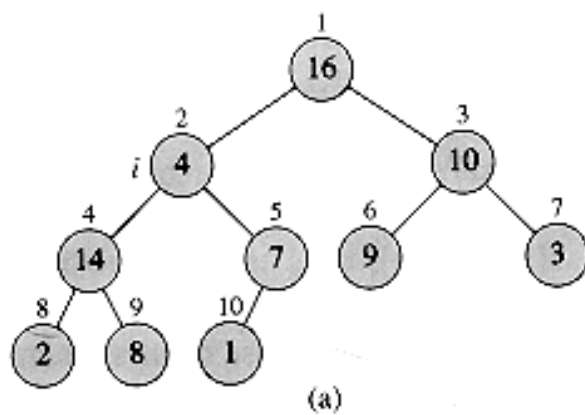
Max-Heapify(A, i)

- Input: Un array A ed un indice i
- Gli alberi Binari con radici $LEFT(i)$ e $RIGHT(i)$ sono Max-Heap
- E' possibile che sia minore di $A[Left(i)]$ e di $A[Right(i)]$.
- Scopo: Ripristinare la proprietà di Max-Heap sul sottoalbero con radice i , facendo scendere l' elemento $A[i]$ nell' array

Max-Heapify(A, i)

1. $l \leftarrow \text{Left}[i]$
2. $r \leftarrow \text{Right}[i]$
3. **if** $A[i] < A[l]$ **and** $l \leq A.\text{heapsize}$
4. **then** $\text{max} \leftarrow l$
5. **else** $\text{max} \leftarrow i$
6. **if** $A[\text{max}] < A[r]$ **and** $r \leq A.\text{heapsize}$
7. **then** $\text{max} \leftarrow r$
8. **if** $\text{max} \neq i$
9. **then** $\text{scambia } A[i] \leftrightarrow A[\text{max}]$
10. **Max-Heapify**(A, max)



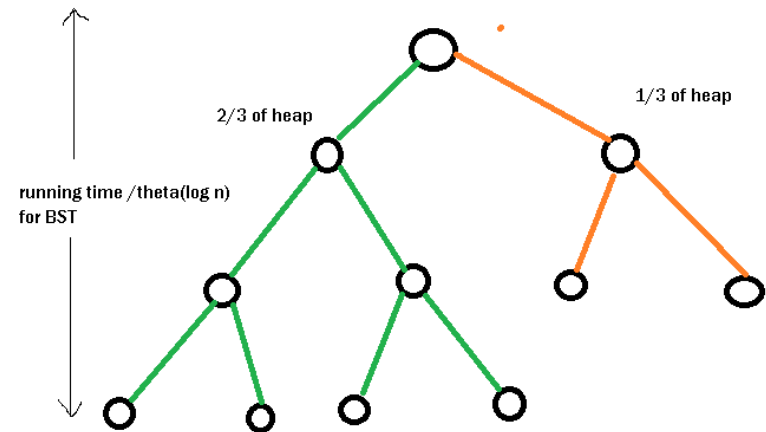


Tempo di Esecuzione MAX-HEAPIFY

- Il tempo di esecuzione Max-HEAPIFY in un sottoalbero di dimensione n con radice in un nodo i è pari al tempo $\Theta(1)$ per le prime istruzioni più il tempo necessario per eseguire il tempo MAX-HEAPIFY in un sottoalbero con radice in uno dei figli del nodo i . I sottoalberi dei figli hanno ciascuno una dimensione (ossia numero dei nodi) che non supera, nel caso peggiore, i $\frac{2}{3}$ dell' intero albero.

Caso Peggior

- Il caso peggiore si ha quando il sottoalbero sinistro è completo al livello h ed il sottoalbero destro è completo al livello $h - 1$. Pertanto il sottoalbero sinistro ha $2^{h+1} - 1$ nodi.



(cont.)

- Ma i nodi totali sono: nodi sottoalbero sinistro + nodi sottoalbero destro + 1.
- Ossia: $(2^{h+1} - 1) + (2^h - 1) + 1 = 3 \cdot 2^h - 1$.
- Quindi il rapporto tra i nodi del sottoalbero sinistro e quelli totali è: $\frac{2 \cdot 2^h - 1}{3 \cdot 2^h - 1} \leq \frac{2}{3}$

Complessità di Max-Heapify

- È descritta dalla ricorrenza:
$$T[n] \leq T(2n/3) + \Theta(1)$$
- La complessità usando il master teorema delle ricorrenze (caso 2) è: $T[n] = \Theta(\log n)$

Complessità Computazionale

- Ricorrenza **Heapify**

$$T(n) = T\left(\frac{2}{3}n\right) + \Theta(1)$$

- Master Teorema delle Ricorrenze

if $T(n) = aT(n/b) + \Theta(n^p)$

dove $p = \log_b a$ then $T(n) = \Theta(\log n \, n^p)$

- Poiché $a = 1$ e $b = 3/2$ allora $p = 0$
- La complessità computazionale di **Heapify** è $\Theta(\log n)$.